

Agent Reshapes the New Paradigm of the Software Industry

Jason Li (李建忠)

Singularity Research / CSDN

Email: lijz@csdn.net

LinkedIn:

<https://www.linkedin.com/in/jianzhongli>





Jason Li

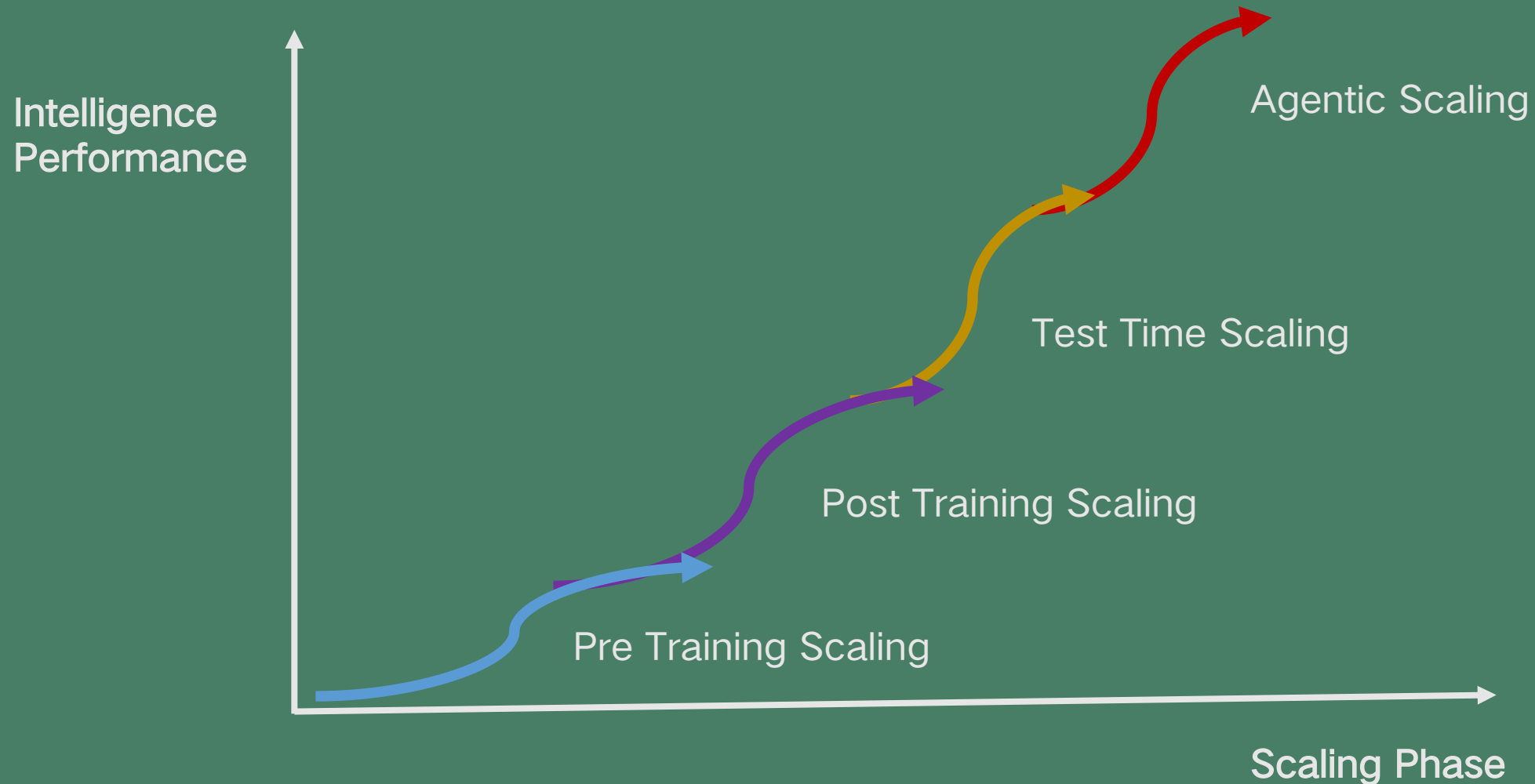
President at Singularity Research, SVP of CSDN

Chairman of Singularity Intelligence Technology Summit (SITS) , and member of the ISO-C++ Standard Committee. He has rich experience and in-depth research in artificial intelligence, software architecture and product innovation. In recent years, he has mainly focused on the application of artificial intelligence methods centered around Large Language Models. He proposed the "ParaShift Cube" for technological product innovation. He is also a serial entrepreneur who has founded well-known technology companies including SoftCompass, Slideldea, and Boolan.



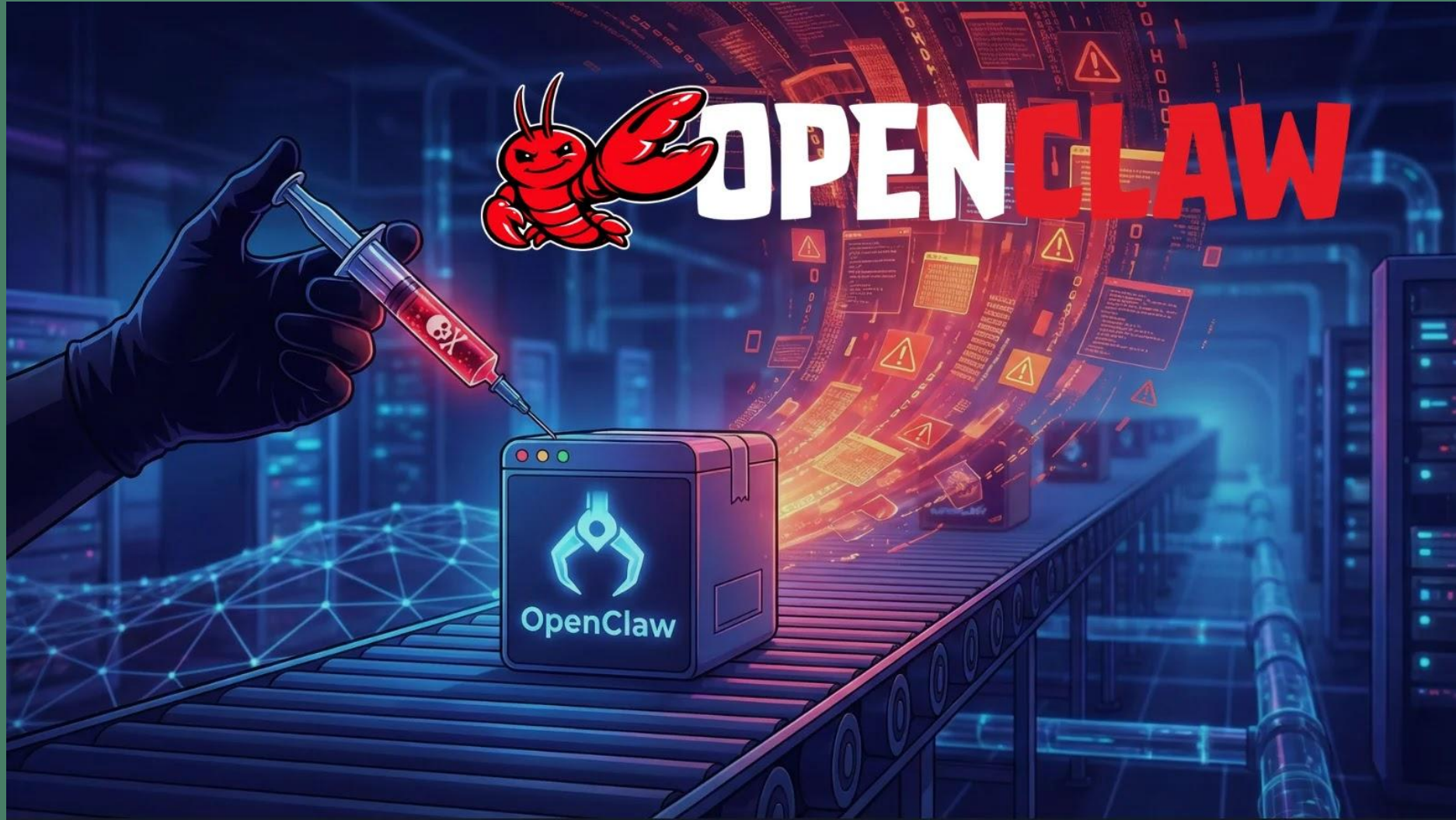
Agentic Scaling : the 4th Paradigm of Scaling Law

GOSIM Paris 2026



Scaling Paradigm	Core Mechanism	Computation Phase	Scaling Dimension	Scaling Objective
Pre-training Scaling	Larger models + More data	Training phase	Parameter count / Data volume	Representation compression
Post-training Scaling	RLHF、SFT	Post-training fine-tuning phase	Preference data / Reward signal strength	Behavior Alignment
Test-time Scaling	Inference-time thinking (COT)	Single inference process	Number of thinking steps / Sampling width	Depth of reasoning
Agentic Scaling	Multi-step, cross-system, self-loop execution	Continuously action sequences at runtime	Number of agents $\times$ Iteration depth $\times$ Time span	Task completion rate





Every generation of technology revolution brings
dual disruption to developers.

1. Disrupts the form of applications: **What**
to DO
2. Disrupts the way of development : **How to**



What changes do Agents bring to software?

GOSIM Paris 2026

	Internet	AI
Application Paradigm	Web (HTML/CSS/JS)	AI Agent
Development Paradigm	Cloud Native	AI Native



Agent Changes Software Application Paradigm



User Interface

Agents will replace traditional software as the primary interface for human interaction.

Downstream Tools

Traditional software will move downstream, becoming tools invoked by Agents.

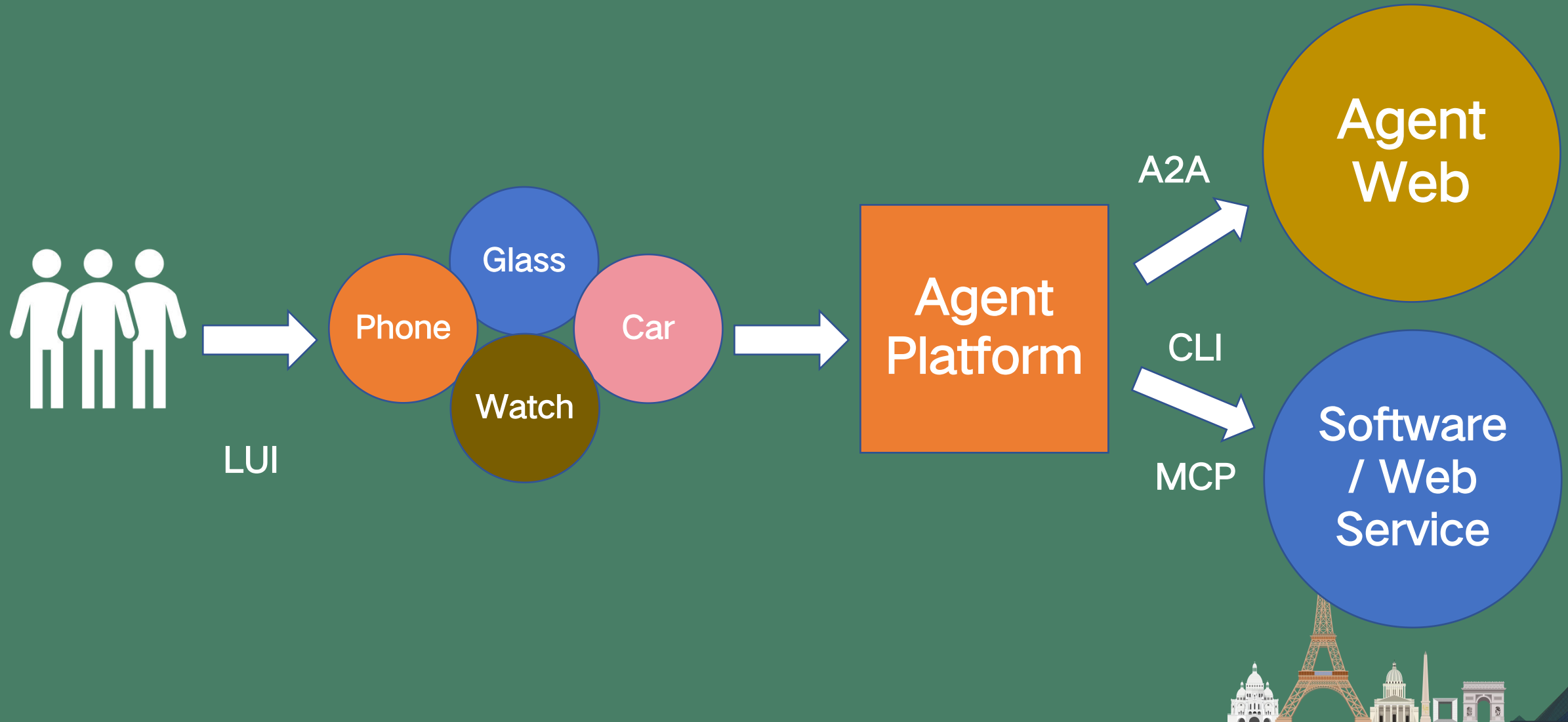
Design Refactoring

Design software for Agents, not for Humans. CLI is more Agent-friendly than GUI.



The Evolution of Interaction Driven by Agents

GOSIM Paris 2026





Build the software tools and APIs required for the Agent's execution chain.



Develop task-specific skills for a dedicated Agent.

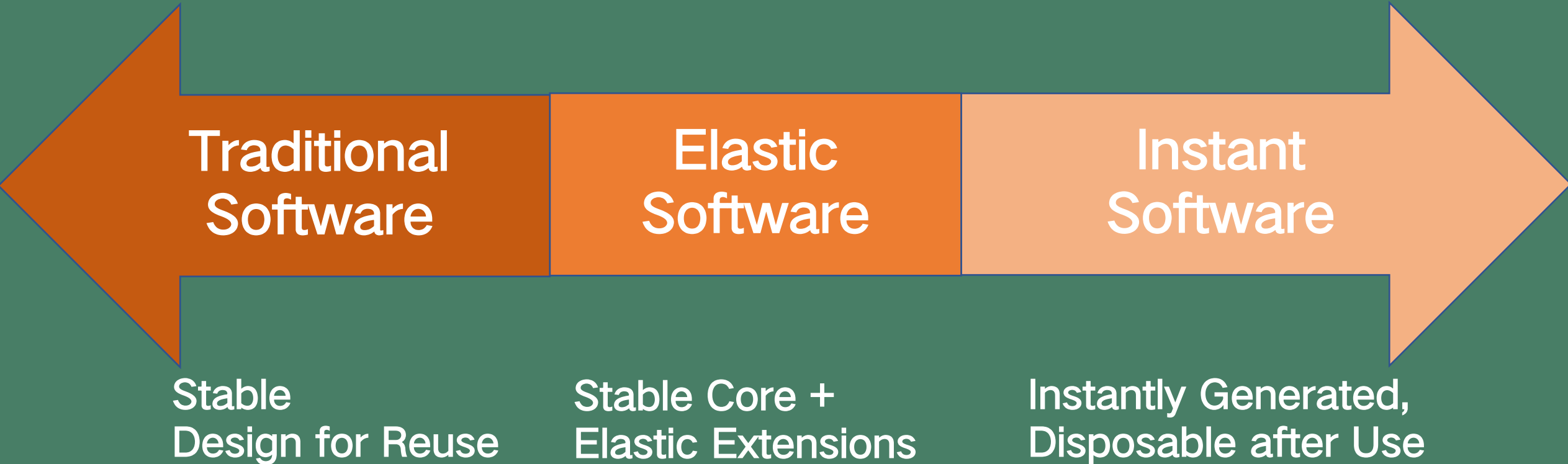
Skills = Job Description
+ SOP + Toolkit



Create environment (cloud/client) for Agent, with sandbox permissions.

Environment = System +
Permissions





A Comparison of Three Software Paradigms

GOSIM Paris 2026

	Traditional Software	Elastic Software	Instant Software
Reuse Form	Stable source code (binary)	Stable source code (binary) + Skills	Skills
Software Engineering	Follows traditional SDLC engineering	Traditional SDLC system + Runtime engineering system	Abandons SDLC, adopts runtime engineering system
AI Development	Spec-driven	Spec + Vibe Coding	Vibe Coding
Capability Binding	Development phase: full definition & implementation Runtime phase: executes pre-compiled deterministic rules	Development phase: stable, high-certainty core capabilities Runtime phase: highly variable, context-dependent capabilities dynamically generated by Agents	Runtime phase: dynamically generated and executed by Agents based on tasks and Skills



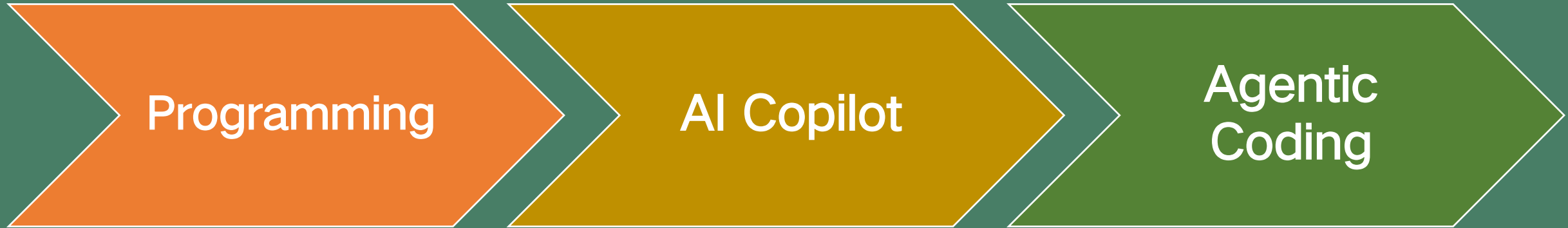


- Instant Software refers to software that is automatically generated by Agents, or "black-box created" by ordinary users using "natural language programming".
- Just like Web programming in the Internet era, Instant Software represents an incremental market and creates a new paradigm for software in the AI era.



Agent Changes Software Development Paradigm





AI-Native Software Development Maturity Model

GOSIM Paris 2026

DIMENSION	L1 Assisted Productivity Unlock Individual Potential	L2 Domain Integration Build Organizational Intelligence	L3 Agent Collaboration Multiply Team Effectiveness	L4 Autonomous Agents Enable Autonomous Delivery	L5 Self-Evolving Factory Drive Continuous Evolution
Infrastructure	- Call foundation models through public cloud APIs - Traditional cloud resources, no dedicated AI Infra - Lack of a unified model management platform	Hybrid deployment with privatized domain models - Dedicated GPU / NPU compute resource pool - Digitized storage of domain knowledge - Unified API Gateway	Agent runtime environment and version management - Long-running Agent session management - Domain Harness configuration management platform	Multi-Agent autonomous orchestration platform and scheduling engine AgentOps decision-chain tracing and token monitoring Token optimization infrastructure (Caching / Routing)	Self-evolving flywheel infrastructure: automated evaluation and online optimization feedback loop - Intelligent cost-benefit optimization platform - Adaptive resource elasticity and cost optimization engine
Knowledge Engineering	- Use only public knowledge, with no domain integration - Documents are unstructured; retrieval is mainly manual - No engineered knowledge management approach	Domain knowledge base development Context Engineering practices initiated - Case library and experience-capture mechanism	- Domain Harness knowledge system construction - Domain Skill library and quality management system - Real-time expert knowledge injection and lesson capture	Knowledge flywheel: multi-Agent sharing and contribution - Agents possess manager-level knowledge synthesis capability - Agent self-optimization experience is automatically captured	Self-evolving knowledge ecosystem: autonomously identifies blind spots and expands itself - Cross-task knowledge abstraction and hierarchical domain knowledge systems Automatic conversion from knowledge to capability and innovation generation
Process & Tools	- Embed AI-assisted tools into traditional workflows - Processes are human-led; AI tools are optional Vibe Coding used exploratorily	- AI integrated into key DevOps stages Intelligent code review and test generation - AI-based code quality baselines and technical debt management	- Adaptive process standards (Spec-driven) - Domain Harness Engineering practices - Agents take on explicit R&D roles; performance and token efficiency are measured	AgentOps and VPT metrics drive continuous optimization - Agents possess autonomous design and planning capabilities - Multi-Agent autonomous orchestration, collaboration, and token optimization	End-to-end delivery flywheel pipeline - Four-layer flywheel path: single task → cross-task → end-to-end → self-evolution - Flywheel self-evolution and automatic discovery of configuration blind spots
Organization & Talent	- Traditional organization plus AI tool training - Basic Prompt Engineering skills - Individual exploration, with no systematic capability building	AI CoE rollout practices and training system - Developers learn Context Engineering - AI usage efficiency is included in measurement	- Role transformation: Harness Engineer, etc. - Developers transition into Agent collaboration supervisors - Human-AI collaboration norms and evaluation standards	Agent Orchestrator / Evaluator as new roles - Organizational operations approach an automated software factory - Evaluation based on value creation and token efficiency	Flywheel Architect: designer and guardian of the flywheel - Networked ecosystem organization where humans and AI evolve together - Innovation incubation platform: complete production and innovation teams formed by micro-teams (super-individuals) + multi-Agent systems
Security & Governance	- Basic AI usage rules and prohibited items - AI content subject to manual review - Simple logging, with no systematic risk assessment	Governance framework including access control and audit AI Gateway and data masking - AI code security scanning and compliance checks	Agent permission boundaries defined and enforced - Harness-layer security design and sandbox isolation - Complete traceable audit of Agent behavior	Permission matrix and monitoring under autonomous orchestration scenarios Cognitive debt assessment and control - Security constraints for Agent self-optimization and automatic rollback	Autonomous governance ecosystem, with AI participating in supervision - AI trustworthiness rating and impact assessment Human-AI co-governance, with humans retaining ultimate control



AI-Native Software Development Maturity Model

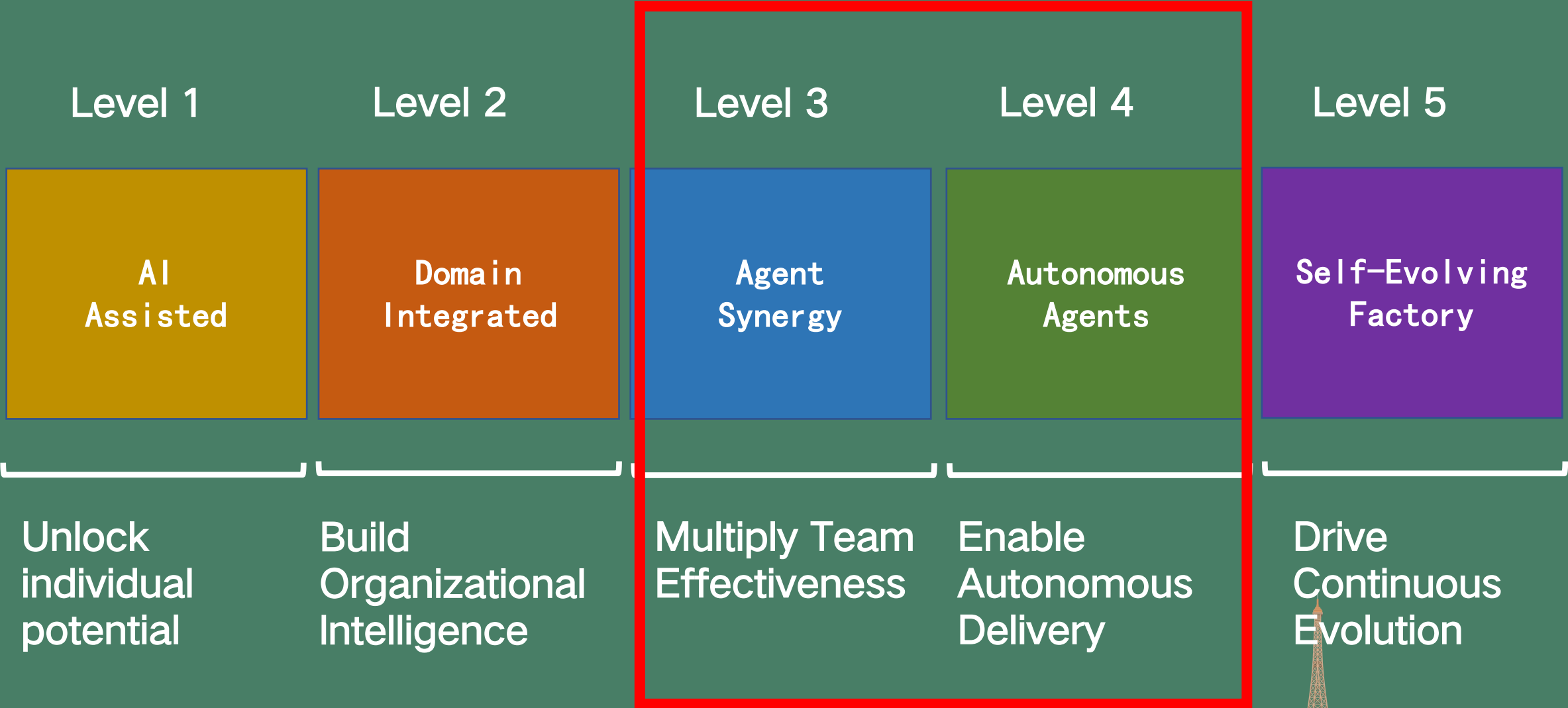
AISMM White Paper

Released by Singularity Research



Five Stages of AISMM Maturity

GOSIM Paris 2026



1. AI Software Infra

2. Knowledge Engineering

3. Process & Tools

4. Organization & Talent

5. Governance & Security



Human in
the Loop



Human out of
the Loop

Human-Agent Synergy

Autonomous Agents



Prompt Engineering

(2022-2023)

Focus on how to clearly articulate requirements to the model in multi-turn dialogues.

Context Engineering

(2024-2025)

Focus on how to provide just the right information in long-horizon human-AI collaborative tasks.

Harness Engineering

(2026-...)

Focus on how to build a closed-loop system that enables Agents to perform tasks reliably, safely and collaboratively.



The Core Elements of Harness Engineering

GOSIM Paris 2026



OpenClaw

Hermes Agent

AutoHarness



Deep Restructuring of Cloud Computing Service Models

GOSIM Paris 2026

IaaS -> Token Infra

IaaS is evolving from hardware resource leasing into infrastructure services for Token factory

PaaS -> MaaS

PaaS is transforming from an application runtime platform into a composable AI capability platform: MaaS + Agent Runtime.

SaaS -> AaaS

SaaS is evolving from a function delivery platform into an agent service platform centered on task delivery — Agent-as-a-Service (AaaS).



12 Frontier Trends of AI Industry Transformation in 2026



Interaction Reshapes New Software Forms

Trend 01.

Agents Become the First User Entry Point, While Traditional Software Moves Downstream

Trend 02.

Generative UI (GenUI) Creates More Delightful Personalized User Experiences

Trend 03.

Agents Refactor the Internet from an Information Network into an Action Network

Trend 04.

Natural Language + Agent-Centric Networks Become the Human-Machine Interface Layer Across Devices



Development Paradigm: From Code Generation to Flywheel Construction

GOSIM Paris 2026

Trend 05.

Instant Software Grows Rapidly, While Elastic Software Reaches Engineering Maturity

Trend 06.

Harnesses Mature from Manual Configuration to AI-Assisted Generation and Automated Optimization

Trend 07.

Development Tools and Infrastructure Become Agent-Native, Not Human-Centric

Trend 08.

The Boundary of Autonomous Software Engineering Keeps Expanding



AI Infrastructure

Trend 09.

Deep Restructuring of Cloud Service Models—from IaaS/PaaS/SaaS to Token Factory/MaaS/AaaS

Trend 10.

Token Economics Becomes Infrastructure Engineering, Not Application-Layer Hacking

Trend 11.

Pooling, Heterogeneity, and Elastic Scheduling of Inference Compute

Trend 12.

Edge-Cloud Collaborative AI Architectures Reach Engineering Maturity



Thanks!

Email: lijz@csdn.net

Linkedin: <https://www.linkedin.com/in/jianzhongli>

